

GastroOS — Documento Técnico Detallado del Sistema

Fecha: 1 de Marzo, 2026
Proyecto: GastroOS — Sistema POS para negocios de comida
Repositorio: `gastro-os`

1. Resumen Ejecutivo

GastroOS es un sistema **POS (Punto de Venta) SaaS multi-tenant** diseñado para negocios de comida (taquerías, pizzerías, cafeterías, comida rápida, y otros). Corre como una aplicación **Next.js 16** desplegada en contenedores Docker vía **Dokploy**, con **Supabase** como backend completo (base de datos PostgreSQL, autenticación, Realtime, y Row Level Security).

El sistema permite a múltiples negocios independientes operar con aislamiento total de datos, gestionar órdenes en tiempo real, controlar inventario automáticamente, administrar cajas registradoras, y recibir notificaciones sonoras en cocina.

2. Stack Tecnológico Completo

2.1 Dependencias de Producción

Paquete	Versión	Propósito
next	16.1.6	Framework React full-stack (App Router, Server Components, Server Actions)
react	19.2.3	Librería UI con Hooks
react-dom	19.2.3	Renderizado DOM para React
@supabase/supabase-js	^2.95.3	SDK de Supabase para JavaScript (queries, auth, realtime)
@supabase/ssr	^0.8.0	Adaptador Supabase para Server-Side Rendering (manejo de cookies y sesiones)

2.2 Dependencias de Desarrollo

Paquete	Versión	Propósito
tailwindcss	^4	Framework CSS utility-first
@tailwindcss/postcss	^4	Plugin PostCSS para TailwindCSS
typescript	^5	Lenguaje tipado estático
@types/node	^20	Tipos TypeScript para Node.js
@types/react	^19	Tipos TypeScript para React
@types/react-dom	^19	Tipos TypeScript para React DOM
eslint	^9 + eslint-config-next 16.1.6	Linter de código

2.3 Servicios Externos

Servicio	Propósito
Supabase (proyecto: cpicczdzemusajdftvtt)	BaaS completo: PostgreSQL, Auth, Realtime, RLS, RPC
Vercel (proyecto: gastro-os, org: team_Gxh7hcNL8vVrdPVUndTh)	Historialmente asociado (.vercel/project.json presente)
Dokploy	Plataforma de despliegue actual via Docker

3. Arquitectura del Sistema

3.1 Diagrama de Componentes

```
graph TB
  subgraph "Cliente (Browser)"
    A["Next.js App (React 19)"]
    B["Supabase Client SDK"]
    C["Web Audio API"]
  end

  subgraph "Servidor (Docker Container)"
    D["Next.js Server (Node.js 22)"]
    E["Middleware (Session Refresh)"]
    F["Server Components"]
    G["API Routes"]
  end

  subgraph "Supabase Cloud"
    H["PostgreSQL DB"]
    I["Auth (JWT)"]
    J["Realtime (WebSockets)"]
    K["RLS Policies"]
    L["RPC Functions"]
  end

  A --> B
  A --> C
  B --> J
  B --> H
  D --> E
  E --> I
  F --> H
  G --> H
  H --> K
  H --> L
```

3.2 Flujo de Datos General

- 1. **Usuario abre la app** → Next.js sirve la página (Server Component o Client Component)
- 2. **Middleware intercepta** cada request → refresca la sesión de Supabase via cookies
- 3. **Server Components** hacen queries directas a Supabase con `getClient()` del servidor
- 4. **Client Components** usan el SDK del browser para queries en tiempo real y mutaciones
- 5. **Supabase RLS** filtra automáticamente todos los datos por `business_id` del usuario
- 6. **Realtime** envía cambios (INSERT/UPDATE en `orders`) al KDS de cocina via WebSockets

4. Contenedor Docker

4.1 Estrategia Multi-Stage Build

El `Dockerfile` usa **3 etapas** para optimizar el tamaño de la imagen:

Stage 1: deps	→ node:22-alpine + npm ci (instala dependencias)
Stage 2: builder	→ Copia deps + código fuente + ejecuta npm run build
Stage 3: runner	→ Solo copia lo necesario para producción

4.2 Detalles Técnicos del Dockerfile

Aspecto	Detalle
Base image	node:22-alpine (mínima, ~5MB)
libc6-compat	Instalado para compatibilidad con binarios nativos
Build args	NEXT_PUBLIC_SUPABASE_URL y NEXT_PUBLIC_SUPABASE_ANON_KEY — inyectadas como ARG de Docker por Dokploy en tiempo de build
Output mode	standalone (en next.config.ts) — genera un server.js auto-contenido
Usuario	nextjs:nodejs (UID/GID 1001) — por seguridad, no corre como root
Puerto	3000, hostname 0.0.0.0
Comando	node server.js
NODE_ENV	production

4.3 Variables de Entorno

Variable	Tipo	Descripción
NEXT_PUBLIC_SUPABASE_URL	Build-time (público)	URL del proyecto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Build-time (público)	Clave anónima para el SDK del browser
SUPABASE_SERVICE_ROLE_KEY	Runtime (secreto)	Clave con permisos elevados para operaciones de servidor

*[!IMPORTANT] Las variables `NEXT_PUBLIC_*` deben estar disponibles en **tiempo de build** (no solo runtime) porque Next.js las embebe directamente en el bundle del cliente.*

4.4 Despliegue via Dokploy

El flujo de despliegue es:

1. **Push a GitHub** → Dokploy detecta cambios
2. **Docker build** → Dokploy ejecuta `docker build` inyectando las build args
3. **Deploy** → El contenedor resultante se ejecuta en el servidor con dominio propio
4. **Dominio** → Configurado en Dokploy con SSL automático (Let's Encrypt)

5. Supabase — Backend as a Service

5.1 Tres Helpers de Conexión

GastroOS implementa **3 clientes de Supabase distintos** según el contexto:

5.1.1 Cliente del Browser (`src/lib/supabase/client.ts`)

```
// Usa createBrowserClient de @supabase/ssr
// Se usa en Client Components ('use client')
// Cookie-based auth session management
```

Uso: Queries en tiempo real, mutaciones desde el frontend, suscripciones Realtime.

5.1.2 Cliente del Servidor (`src/lib/supabase/server.ts`)

```
// Usa createServerClient de @supabase/ssr
// Lee cookies del request (server-side)
// Seguro para Server Components y Server Actions
```

Uso: Validar autenticación en layouts, cargar datos iniciales, verificar membresías.

5.1.3 Middleware (`src/lib/supabase/middleware.ts`)

```
// Intercepta CADA request al servidor
// Refresca tokens JWT cuando están por expirar
// Redirige a /login si no hay sesión (excepto rutas públicas)
```

Rutas públicas (no requieren auth): `/login` , `/register` , `/auth/callback` , `/api/*`

5.2 Autenticación

- **Motor:** Supabase Auth (JWT)
- **Persistencia:** Cookies HTTP-only (manejadas por `@supabase/ssr`)
- **Registro:** Formulario email + password → creación en Supabase Auth
- **Login:** Email + password
- **Middleware:** Refresca tokens en cada request y redirige a `/login` si no hay sesión

5.3 Realtime (WebSockets)

GastroOS usa **Supabase Realtime** para la **pantalla de cocina (KDS)**:

- **Canal:** kitchen-orders
- **Eventos escuchados:**
 - `INSERT` en tabla `orders` (filtrado por `business_id`)
 - `UPDATE` en tabla `orders` (filtrado por `business_id`)
- **Al recibir INSERT:** dispara sonido + animación visual
- **Al recibir UPDATE:** recarga las órdenes (debounce 300ms)
- **Fallback:** Si Realtime falla → polling cada 10 segundos

6. Base de Datos — Esquema Completo

6.1 Tablas Principales (13)

```
erDiagram
    businesses ||--o{ business_memberships : "tiene"
    businesses ||--o{ categories : "tiene"
    businesses ||--o{ products : "tiene"
    businesses ||--o{ orders : "tiene"
    businesses ||--o{ cash_registers : "tiene"
    businesses ||--o{ inventory_items : "tiene"
    businesses ||--o{ expenses : "tiene"
    businesses ||--o{ audit_logs : "tiene"
    businesses ||--|| folio_sequences : "tiene"
    businesses ||--|| subscriptions : "tiene"

    orders ||--o{ order_items : "contiene"
    orders ||--o{ payments : "tiene"

    cash_registers ||--o{ payments : "recibe"
    cash_registers ||--o{ cash_movements : "tiene"

    inventory_items ||--o{ inventory_movements : "tiene"
    inventory_items ||--o{ product_recipes : "usado en"
    products ||--o{ product_recipes : "tiene"
    products }o--|| inventory_items : "vinculado a"

    plans ||--o{ subscriptions : "asignado"
```

6.2 Detalle de Cada Tabla

Tabla	Propósito	Campos Clave
businesses	Negocio/tenant	name, type (taqueria/pizzeria/cafeteria/fast_food/other), operation_mode (counter/restaurant), límites (products, orders_day, users, storage_mb), default_keep_float_amount, cash_difference_threshold
business_memberships	Relación usuario↔negocio	user_id, business_id, role (OWNER/ADMIN/CASHIER/KITCHEN/INVENTORY), status

categories	Categorías de productos	name, position, active
products	Catálogo de productos	name, price, description, image_url, has_recipe, active, inventory_item_id (vínculo directo a inventario), soft-delete via deleted_at
orders	Órdenes del POS	folio (formato GOS-000001), status, service_type (dine_in/takeaway/delivery), table_number, subtotal_snapshot, discount_amount, discount_reason, tax_snapshot, total_snapshot, notes, cancel_reason
order_items	Items por orden	name_snapshot, price_snapshot (congelados al crear), quantity, notes
payments	Pagos procesados	amount, method (cash/card/transfer), status (pending/paid/refunded/void), cash_register_id
cash_registers	Turnos de caja	status (open/closed), opening_amount, expected_cash, counted_cash, difference, keep_float_amount, withdrawn_cash, summary_snapshot (JSONB immutable), requires_review
cash_movements	Entradas/salidas de caja	type (in/out), amount, reason
inventory_items	Artículos de inventario	name, unit (pz/paquete/caja/litro/kg/g/ml), stock_current, stock_min, track_mode (manual/auto)
inventory_movements	Historial de movimientos	type (manual_adjustment/purchase/auto_sale/waste), delta, ref_entity_type, ref_entity_id
expenses	Gastos del negocio	category, description, amount
audit_logs	Log de auditoría (immutable)	action, entity (order/payment/cash_register/cash_movement/inventory/product), entity_id, metadata (JSONB)
folio_sequences	Secuencia de folios por negocio	last_folio (auto-incremento vía función SQL)
plans	Catálogo de planes SaaS	slug, name, price, billing_interval, features (JSONB con límites)
subscriptions	Suscripción activa por negocio	plan_id, status (trialing/active/past_due/expired/canceled), trial_end, plan_code_snapshot, price_snapshot
profiles	Perfil de usuario	accepted_terms_at, accepted_terms_version

6.3 Planes SaaS

Plan	Slug	Precio	Productos	Órdenes/día	Usuarios	Almacenamiento
------	------	--------	-----------	-------------	----------	----------------

Demo (5 días)	demo	\$0 MXN	100	200	3	50 MB
Básico	basic	\$69 MXN/mes	10	100	2	25 MB
Premium Mensual	premium_monthly	\$120 MXN/mes	500	1,000	10	200 MB
Premium Anual	premium_annual	\$1,200 MXN/año	500	1,000	10	200 MB

6.4 Row Level Security (RLS)

Todas las tablas tienen RLS habilitado. La política base `tenant_isolation` asegura que cada usuario solo puede ver datos del negocio al que pertenece:

```
-- Patrón repetido en TODAS las tablas:
CREATE POLICY "tenant_isolation" ON [tabla]
FOR ALL USING (
  business_id IN (
    SELECT business_id FROM business_memberships
    WHERE user_id = auth.uid()
  )
);
```

Políticas especiales:

- `plans` : lectura pública para autenticados, escritura denegada
- `subscriptions` : solo lectura del propio tenant, escritura denegada (solo vía RPC/admin)
- `businesses` : permite INSERT a cualquier autenticado (para onboarding)
- `business_memberships` : permite INSERT solo si `user_id = auth.uid()`

6.5 Triggers e Integridad de Datos

Trigger	Tabla	Evento	Función
<code>orders_updated_at</code>	<code>orders</code>	BEFORE UPDATE	Actualiza <code>updated_at</code> automáticamente
<code>orders_require_payment_for_close</code>	<code>orders</code>	BEFORE UPDATE	Impide cerrar orden sin pago pagado
<code>payments_require_open_register</code>	<code>payments</code>	BEFORE INSERT	Valida que la caja esté abierta
<code>payments_refund_void_check</code>	<code>payments</code>	BEFORE UPDATE	Solo permite refund/void si el pago estaba 'paid'
<code>trg_auto_inventory_on_payment</code>	<code>payments</code>	AFTER INSERT (when status='paid')	Auto-descuenta inventario atómicamente
<code>subscriptions_updated_at</code>	<code>subscriptions</code>	BEFORE UPDATE	Actualiza <code>updated_at</code>

6.6 Funciones RPC (SECURITY DEFINER)

Función	Descripción
get_next_folio(business_id)	Genera folio secuencial G05-000001, G05-000002, etc. Usa upsert atómico
apply_inventory_movement(...)	Movimiento de inventario atómico con: lock del item, validación cross-tenant, RBAC por rol, actualización de stock, registro de movimiento, auditoría. Todo en una transacción
auto_deduct_inventory_on_payment()	Trigger que al recibir pago 'paid' itera los items de la orden, descuenta del inventario (solo items en modo auto), y registra movimientos + auditoría. Idempotente
create_business_and_owner_membership(...)	Crea negocio + membresía OWNER + subscription trial de 5 días en una transacción atómica
get_subscription_status(business_id)	Retorna estado de suscripción. Si trial venció, lo marca como 'expired' automáticamente
is_admin()	Verifica si el usuario es admin de la plataforma

7. Estructura de la Aplicación

7.1 Mapa de Rutas

src/app/	
├─ page.tsx	→ Raíz (redirige)
├─ layout.tsx	→ Layout global (metadata, fonts)
├─ globals.css	→ Estilos globales (~180KB)
├─ (auth)/	→ Grupo de rutas de autenticación
└─ layout.tsx	→ Layout sin sidebar
└─ login/page.tsx	→ Página de login
└─ register/	→ Registro (2 archivos)
└─ onboarding/page.tsx	→ Wizard para crear negocio
├─ auth/	
└─ callback/	→ OAuth callback de Supabase
├─ dashboard/	→ Área principal (protegida)
└─ layout.tsx	→ Layout con Sidebar + guards
└─ page.tsx	→ Dashboard principal (estadísticas)
└─ pos/	→ Punto de Venta (1,297 líneas)
└─ kitchen/	→ Pantalla de Cocina (KDS)
└─ orders/	→ Listado de órdenes
└─ cash/	→ Gestión de caja registradora
└─ menu/	→ CRUD de productos y categorías
└─ inventory/	→ Inventario con movimientos
└─ analytics/	→ Analítica de ventas
└─ reports/	→ Reportes avanzados

		└─ audit/	→ Log de auditoría
		└─ settings/	→ Configuración y equipo
		└─ components/	→ Sidebar, SubscriptionGuard, UpdateFlyer
		└─ admin/	→ Panel Super-Admin (plataforma)
		└─ layout.tsx	→ Layout admin (verifica is_admin)
		└─ page.tsx	→ Redirige a /admin/businesses
		└─ businesses/	→ Gestión de negocios (SaaS)
		└─ billing/	
		└─ upgrade/	→ Página de upgrade de plan
		└─ account/	→ Estados de cuenta
		└─ blocked/	→ Cuenta bloqueada
		└─ suspended/	→ Cuenta suspendida
		└─ terms/	
		└─ accept/	→ Aceptación de términos
		└─ api/	→ API Routes
		└─ team/	→ Gestión de equipo (invitaciones)
		└─ test-inventory/	→ Endpoint de prueba de inventario

7.2 Flujo de Protección del Dashboard Layout

Cuando un usuario accede a cualquier ruta bajo `/dashboard`, el **Server Component** `DashboardLayout` ejecuta **5 verificaciones** en cadena:

```

flowchart TD
    A["Usuario accede a /dashboard/*"] --> B{"¿Tiene sesión?"}
    B -->|No| C["→ /login"]
    B -->|Sí| D{"¿Aceptó términos v2026-02-12?"}
    D -->|No| E["→ /terms/accept"]
    D -->|Sí| F{"¿Tiene membresía activa?"}
    F -->|No| G{"¿Es admin de plataforma?"}
    G -->|Sí| H["→ /admin"]
    G -->|No| I["→ /onboarding"]
    I --> J{"¿Negocio eliminado soft-delete?"}
    J -->|Sí| K["→ /account/blocked"]
    J -->|No| L{"¿Suscripción activa?"}
    L -->|No, suspended| M["→ /account/suspended"]
    L -->|No, expired| N["→ /billing/upgrade"]
    L -->|Sí| O["✅ Renderiza Dashboard"]
  
```

Adicionalmente, el componente **SubscriptionGuard** (invisible) monitorea cada **5 segundos** el estado de la suscripción para detectar cambios en tiempo real:

- Si el negocio fue eliminado → redirige a `/account/blocked`
- Si la suscripción fue suspendida → redirige a `/account/suspended`
- Si expiró/canceló → redirige a `/billing/upgrade`

8. Módulos del Dashboard en Detalle

8.1 Punto de Venta (POS) — /dashboard/pos

El componente más grande del sistema (~1,300 líneas). Funcionalidades:

Funcionalidad	Detalle
Selección de productos	Grid de productos filtrable por categoría
Personalización	Modal para agregar notas y modificar cantidad por producto
Tipo de servicio	dine_in (para comer aquí), takeaway (para llevar), delivery (a domicilio)
Descuentos	Monto y razón (solo OWNER/ADMIN)
Enviar a cocina	Crea la orden con folio gos-xxxxxx via RPC get_next_folio y pone status IN_PREP
Gestión de órdenes abiertas	Sidebar lateral muestra órdenes no cerradas con indicador SLA por colores
Retomar órdenes	Permite reabrir una orden para agregar más items
Pago	Soporta efectivo (con cálculo de cambio), tarjeta, y transferencia
Validación de caja	No permite cobrar si no hay caja abierta
Límites de plan	Verifica límite de órdenes/día antes de crear
Impresión de ticket	Componente TicketPreview para generar ticket imprimible
Realtime	Suscripción a cambios para sincronizar órdenes abiertas
Toast notifications	Sistema de notificaciones in-app
Vista mobile	Tabs para navegar entre catálogo, carrito, y órdenes abiertas

8.2 Cocina (KDS) — /dashboard/kitchen

Flujo completo de un pedido entrando a cocina:

1. **Cajero crea orden en POS** → INSERT en tabla `orders` con status `IN_PREP`
2. **Supabase Realtime** envía evento `INSERT` al KDS via WebSocket
3. **Debounce**: el KDS espera 300ms para agrupar múltiples cambios
4. **Recarga de órdenes**: query a Supabase filtrando por `status IN ('OPEN', 'IN_PREP', 'READY')`
5. **Sonido de nuevo pedido**: se ejecuta el `playBeep()` via Web Audio API
6. **Animación visual**: el card de la orden parpadea con clase CSS `kds-order-new` por 3 segundos
7. **KDS muestra**: folio, tiempo transcurrido, items con cantidades y notas, botón de acción

Sistema de Audio Detallado

El sonido de notificación se implementa con la **Web Audio API** nativa del navegador (sin archivos de audio externos):

```
Beep 1: Onda sinusoidal a 800 Hz, volumen 0.3, duración 200ms
      ↓ (pausa de 100ms)
Beep 2: Onda sinusoidal a 1000 Hz, volumen 0.3, duración 200ms
```

Desbloqueo de audio:

- Los navegadores bloquean el audio hasta que el usuario interactúa con la página
- GastroOS intenta `AudioContext.resume()` al cargar la página
- Si falla, registra listeners en `click`, `touchstart`, y `keydown` para desbloquear en la primera interacción
- Si el audio no está habilitado, muestra un botón 🎧 "Activar" en el header

Indicador de conexión Realtime:

-  **En vivo:** WebSocket conectado y funcionando
-  **Reconectando:** Intentando reconectar
-  **Polling:** WebSocket falló → fallback a polling cada 10 segundos

Acciones en Cocina

El cocinero puede **avanzar el estado** de cada orden con un solo botón:

Estado actual	Acción disponible	Color
OPEN	→ En preparación	Azul (primary)
IN_PREP	→ Lista	Amarillo (warning)
READY	→ Entregada	Verde (success)

8.3 Caja Registradora — /dashboard/cash

Funcionalidad	Detalle
Apertura de caja	Ingresa monto inicial de apertura
Movimientos	Entradas (in) y salidas (out) con razón obligatoria
Cierre de caja	Preview con summary snapshot, conteo por denominación, diferencia, monto a dejar como fondo
Summary snapshot	JSONB inmutable que congela ventas por método (cash/card/transfer), entradas, salidas, reembolsos, voids, y warnings
Restricciones	Solo 1 turno abierto por usuario por negocio (unique index)
Auditoría	Todo queda en <code>audit_logs</code>
Configuración	<code>default_keep_float_amount</code> (\$150 MXN por defecto) y <code>cash_difference_threshold</code> (\$20 MXN)

8.4 Menú — /dashboard/menu

- CRUD completo de **categorías** (nombre, posición, activo/inactivo)
- CRUD completo de **productos** (nombre, precio, descripción, imagen, categoría, activo, vínculo a inventario)
- Verificación de **límite de productos** por plan antes de crear
- Soft-delete de productos (preserva historial en órdenes)

8.5 Inventario — /dashboard/inventory

- CRUD de **items de inventario** con unidades (pz, paquete, caja, litro, kg, g, ml)
- Dos modos de seguimiento:
 - **Manual:** ajustes manuales, compras, merma

- **Auto:** descuento automático al pagar una orden (trigger en DB)
- **Movimientos:** adjustment, entry (purchase), waste — todos via RPC `apply_inventory_movement`
- **Indicadores de stock:** normal, bajo ($\text{stock} \leq \text{stock_min}$), sin stock
- **Recetas:** relación many-to-many producto↔inventario (tabla `product_recipes`)

8.6 Analítica — `/dashboard/analytics`

- **Productos más vendidos** (nombre, cantidad, ingresos)
- Dashboard de métricas del negocio

8.7 Auditoría — `/dashboard/audit` (Solo OWNER)

- Log inmutable de todas las acciones: `audit_logs`
- Entidades auditadas: order, payment, cash_register, cash_movement, inventory, product
- Metadata JSONB con detalles contextuales de cada acción

8.8 Equipo — `/dashboard/settings/team` (Solo OWNER)

- Gestión de miembros del equipo
- Asignación de roles
- API route en `/api/team/` para invitaciones

9. Sistema de Roles y Permisos

9.1 Roles Disponibles

Rol	Propósito
OWNER	Dueño del negocio — acceso total
ADMIN	Administrador — casi todo excepto gestión de usuarios y configuración
CASHIER	Cajero — POS, órdenes, caja
KITCHEN	Cocina — solo cambiar estados de órdenes
INVENTORY	Inventario — ajustes, merma, compras, recetas

9.2 Mapa Completo de Permisos

Permiso	OWNER	ADMIN	CASHIER	KITCHEN	INVENTORY
product:create/edit/delete	✓	✓	✗	✗	✗
category:create/edit	✓	✓	✗	✗	✗
cash_register:open/close	✓	✓	✓	✗	✗
cash_movement:create	✓	✓	✓	✗	✗
order:create	✓	✓	✓	✗	✗
order:cancel	✓	✓	✗	✗	✗
order:discount	✓	✓	✗	✗	✗

order:reprint	✓	✓	✓	✗	✗
order:change_status	✓	✓	✗	✓	✗
inventory:adjust/waste	✓	✓	✗	✗	✓
recipe:edit	✓	✓	✗	✗	✓
purchase:register	✓	✓	✗	✗	✓
report:sales	✓	✓	✗	✗	✗
report:inventory	✓	✓	✗	✗	✓
user:create/edit_role	✓	✗	✗	✗	✗
business:config	✓	✗	✗	✗	✗
data:export	✓	✗	✗	✗	✗

10. Máquina de Estados de Órdenes

```
stateDiagram-v2
    [*] --> OPEN : Crear orden
    OPEN --> IN_PREP : Enviar a cocina
    OPEN --> CANCELLED : Cancelar
    IN_PREP --> READY : Marcar lista
    IN_PREP --> CANCELLED : Cancelar
    READY --> DELIVERED : Entregar (modo restaurant)
    READY --> CLOSED : Cerrar directo (modo counter)
    DELIVERED --> CLOSED : Cerrar
    CLOSED --> [*]
    CANCELLED --> [*]
```

Particularidades:

- Modo **counter** (food truck, mostrador): puede saltar de READY → CLOSED directamente
- Modo **restaurant** (servicio a mesa): requiere READY → DELIVERED → CLOSED
- **Cancelar** siempre requiere razón (cancel_reason)
- **Cerrar** requiere un pago paid existente (enforced por trigger)
- **Folio**: formato GOS-000001 , secuencial por negocio

11. Onboarding y Flujo SaaS

11.1 Registro y Onboarding

```
flowchart LR
    A["Registro (email+pass)"] --> B["Aceptar Términos"]
    B --> C["Wizard Onboarding"]
    C --> D["Seleccionar tipo de negocio"]
    D --> E["Seleccionar modo (counter/restaurant)"]
```

```
E --> F["Nombrar negocio"]
F --> G["RPC: create_business_and_owner_membership"]
G --> H["→ Dashboard con trial de 5 días"]
```

Tipos de negocio: Taquería 🌮, Pizzería 🍕, Cafetería ☕, Comida Rápida 🍔, Otro 🎨

La RPC `create_business_and_owner_membership` ejecuta en **una sola transacción**:

- 1. Valida que no tenga un negocio previo
- 2. Crea el registro en `businesses`
- 3. Crea membresía `OWNER + active`
- 4. Inicializa secuencia de folios
- 5. Crea suscripción trial de **5 días**

11.2 Ciclo de Vida de Suscripción

```
stateDiagram-v2
    [*] --> trialing : Onboarding (5 días)
    trialing --> active : Admin asigna plan pagado
    trialing --> expired : Trial vence
    active --> past_due : Pago atrasado
    active --> canceled : Cancelación
    past_due --> active : Pago recibido
    past_due --> suspended : Admin suspende
    expired --> active : Admin asigna plan
    canceled --> [*]
    suspended --> active : Admin reactiva
```

12. Panel de Administración (Super-Admin)

Accesible en `/admin` — solo usuarios marcados como admin via RPC `is_admin` .

Funcionalidad	Descripción
Listado de negocios	Ver todos los negocios con su estado de suscripción
Detalle de negocio	Información completa: plan, miembros, métricas
Asignar plan	Cambiar plan de un negocio y actualizar límites
Desasignar plan	Revertir a sin plan
Suspender/Bloquear	Suspender negocio o bloquear usuarios
Eliminar negocio	Soft-delete con cascada controlada
Insights SaaS	Métricas globales de la plataforma

13. Componentes Clave del Frontend

13.1 Sidebar

- **Desktop:** Sidebar colapsable que se expande al hover, icono + texto, Lucide-style SVG icons
- **Mobile:** Drawer lateral con hamburger button, overlay, y cierre automático al navegar
- **Navegación filtrada por permisos:** cada item verifica `hasPermission(role, permission)`
- **Account header:** clickable, navega a "Mi cuenta y plan"
- **Logout:** con confirmación modal (DialogContext)

13.2 BusinessContext

React Context que proporciona a toda la app:

- `businessId` — ID del negocio actual
- `businessName` — nombre del negocio
- `role` — rol del usuario
- `userId` — ID del usuario
- `loading` — estado de carga

13.3 DialogContext

Sistema de diálogos promise-based:

- `confirm()` → retorna `Promise<boolean>` (Aceptar/Cancelar)
- `alert()` → retorna `Promise<void>` (solo Aceptar)
- Variantes: `danger`, `warning`, `success`, `primary`

13.4 UpdateFlyer

Componente de anuncio de actualizaciones:

- Máximo 3 visualizaciones (persiste en `localStorage`)
- Opción "no volver a mostrar"
- Dismissible

14. Índices de Rendimiento en la Base de Datos

Índice	Tabla	Columnas	Condición
<code>idx_products_business</code>	<code>products</code>	<code>business_id</code>	WHERE <code>deleted_at</code> IS NULL
<code>idx_orders_business_status</code>	<code>orders</code>	<code>business_id</code> , <code>status</code>	WHERE <code>deleted_at</code> IS NULL
<code>idx_payments_cash_register</code>	<code>payments</code>	<code>cash_register_id</code>	WHERE <code>deleted_at</code> IS NULL
<code>idx_order_items_order</code>	<code>order_items</code>	<code>order_id</code>	—
<code>idx_cash_registers_business_status</code>	<code>cash_registers</code>	<code>business_id</code> , <code>status</code>	WHERE <code>deleted_at</code> IS NULL
<code>idx_cash_registers_business_opened</code>	<code>cash_registers</code>	<code>business_id</code> , <code>opened_at</code> DESC	—
<code>idx_audit_logs_business_created</code>	<code>audit_logs</code>	<code>business_id</code> , <code>created_at</code> DESC	—

idx_cash_registers_user_status	cash_registers	opened_by, status	WHERE deleted_at IS NULL
idx_inventory_items_business_active	inventory_items	business_id, active	WHERE deleted_at IS NULL
idx_inventory_movements_business_date	inventory_movements	business_id, created_at DESC	WHERE deleted_at IS NULL
idx_inventory_movements_ref	inventory_movements	ref_entity_type, ref_entity_id	WHERE deleted_at IS NULL
idx_one_paid_per_order (UNIQUE)	payments	order_id	WHERE status='paid' AND deleted_at IS NULL
idx_one_open_register_per_user (UNIQUE)	cash_registers	business_id, opened_by	WHERE status='open' AND deleted_at IS NULL
idx_unique_auto_sale_per_order_item (UNIQUE)	inventory_movements	item_id, ref_entity_id	WHERE type='auto_sale' AND ref_entity_id IS NOT NULL AND deleted_at IS NULL

15. Migraciones SQL (Historial de Evolución)

El directorio `supabase/` contiene **31 archivos SQL** que documentan la evolución del sistema:

Fase	Archivo(s)	Descripción
Inicial	schema.sql, migration.sql	Esquema base + migración inicial
Fase 1	fase-1-verificacion.sql	Verificaciones de integridad
Fase 2	fase-2-implementation.sql, fase-2-cash-closing.sql	Cash register closing system
Fase 3	fase-3-cash-closing-hardening.sql	Hardening (keep float, count breakdown, summary snapshot)
Fase 4	fase-4-production-fixes.sql	Fixes de producción
Fase 5	fase-5-terms.sql	Términos y condiciones
SaaS 1	fase-saas-1-onboarding-rpc.sql	RPC de onboarding atómico
SaaS 2	fase-saas-2-trial-system.sql	Planes, suscripciones, trial de 5 días
SaaS 3	fase-saas-3-plan-limits.sql	Límites por plan
SaaS 4	fase-saas-4-admin-panel.sql	Panel de administración SaaS

Features	add_recipes.sql, add_team_management.sql	Recetas e invitaciones de equipo
Inventory	auto_inventory_on_payment.sql, fix_auto_inventory_ref.sql	Descuento automático de inventario
Fixes	fix-*.sql, hotfix_*.sql, diagnose_*.sql	Correcciones puntuales de RLS, columnas, políticas
Admin	admin-*.sql	Funciones de super-admin (bloqueo, eliminación, insights)

16. Resumen de Conexiones del Sistema

```
graph LR
    subgraph "Entrada"
        U["👤 Usuario (Browser)"]
    end

    subgraph "Capa de Presentación"
        NJ["Next.js 16 (Docker)"]
    end

    subgraph "Capa de Middleware"
        MW["Middleware: Session Refresh"]
        RLS["RLS: Tenant Isolation"]
    end

    subgraph "Capa de Datos"
        DB["PostgreSQL (Supabase)"]
        RT["Realtime (WebSockets)"]
        AUTH["Auth (JWT + Cookies)"]
    end

    subgraph "Capa de Negocio (DB)"
        TR["Triggers (5)"]
        RPC["RPCs (6)"]
        IDX["Indexes (14)"]
    end

    U --> NJ
    NJ --> MW --> AUTH
    NJ --> DB
    NJ --> RT
    DB --> RLS
    DB --> TR
    DB --> RPC
    DB --> IDX
    RT --> U
```

Documento generado el 1 de Marzo de 2026 mediante análisis exhaustivo del código fuente del proyecto
GastroOS.